

1. TITLE PAGE:

Project Title: Diabetes Detection
Course: Build Your Own Project
Student Name: Samriti Sharma
Registration Number: 25BCY10160
Institute: Vellore Institute Of Technology, Bhopal.
Year: 2025

2. INTRODUCTION:

Diabetes is one of the most prevalent chronic diseases across the globe. Early assessment can allow individuals to make preventative changes and avoid serious risk. \ In this project, I implement a basic machine learning model which predicts whether an individual is at risk for diabetes based on various medical characteristics such as glucose level, BMI, insulin level, age, etc.

Therefore, the purpose of this project is to create a foundational predictive model that could supplement a doctor's or patient's assessment as an early warning system.

3. PROBLEM STATEMENT:

Traditionally, clinicians would analyze patient data to ascertain whether a patient is diabetic or not. This is time consuming and possibly subjective, and with thousands of patients, it might not even be practical.

Thus, the problem is:

Is it possible to predict diabetes for a patient automatically from their medical measurements?

4. GOALS:

- To preprocess and clean medical data.
- To fit machine learning models to classify patients as diabetic or not.
- To assess model evaluation metrics: accuracy, precision, recall, F1-score, ROC-AUC.
- To save the final model to use for future predictions.

5. SPECIFICATIONS FOR FUNCTIONALITY:

The dataset needs to be loaded and processed by the system.

The system must manage absent values.⁶

The system needs to train at least one machine learning model.

The system ought to assess the performance of the model.

The system needs to store the top-performing model.

The system ought to deliver a forecast for an example individual

6. NON-FUNCTIONAL REQUIREMENTS:

The package should be easily runnable in a Jupyter Notebook

Ensure there is no clear output and explanation which makes sense at the same time.

The latter should take a couple of seconds to complete.

7. DATASET DESCRIPTION:

The dataset that is used here is the Pima Indians Diabetes Dataset.

It contains the following features:

Pregnancies

Glucose

BloodPressure

SkinThickness

Insulin

BMI

DiabetesPedigreeFunction

Age

Outcome (0 = non-diabetic, 1 = diabetic)

Some values such as 0 in Glucose or BMI are not possible, so they have to be treated as missing and filled properly.

8. METHODOLOGY:

Step 1: Data Loading

I read the data-set into a pandas DataFrame from a CSV file.

Step 2: Data Cleaning

Some of the medical values were "0" when they should not be (e.g., glucose = 0).

I substituted them with NaN and used a median imputer to fill these.

Step 3: Preprocessing

Used a pipeline consisting of:

(Fill missing values with median, retain values filled in the new column - imputer)

StandardScaler (scales all columns)

Step 4: Train-Test Split

I split the data:

80% → training

20% → testing

Applied stratify so two sets have equivalent relative diabetic ratios.

Step 5: Model Training

I trained two models:

Logistic Regression

Random Forest Classifier

We trained both models on the preprocessed data.

Step 6: Model Evaluation

For both models, I calculated:

Accuracy

Precision

Recall

F1 Score

F1 Score

ROC AUC

The better model was chosen based on its ROC AUC.

Step 7: I saved the model.

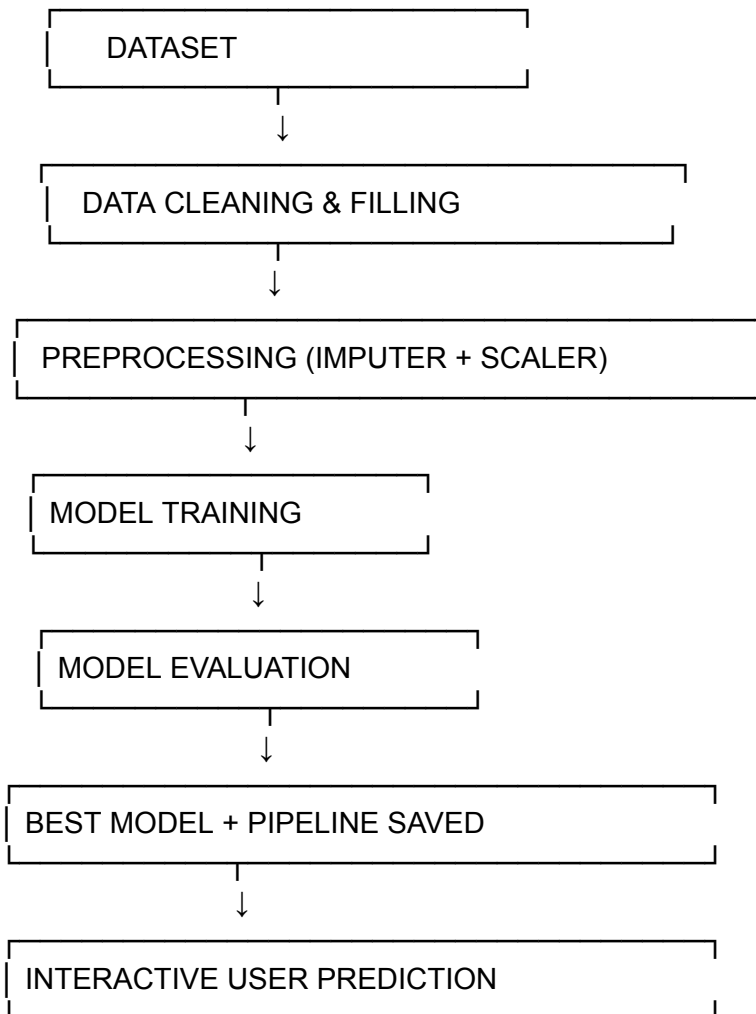
Step 8: Demo Prediction

I created a random patient and tested the model.

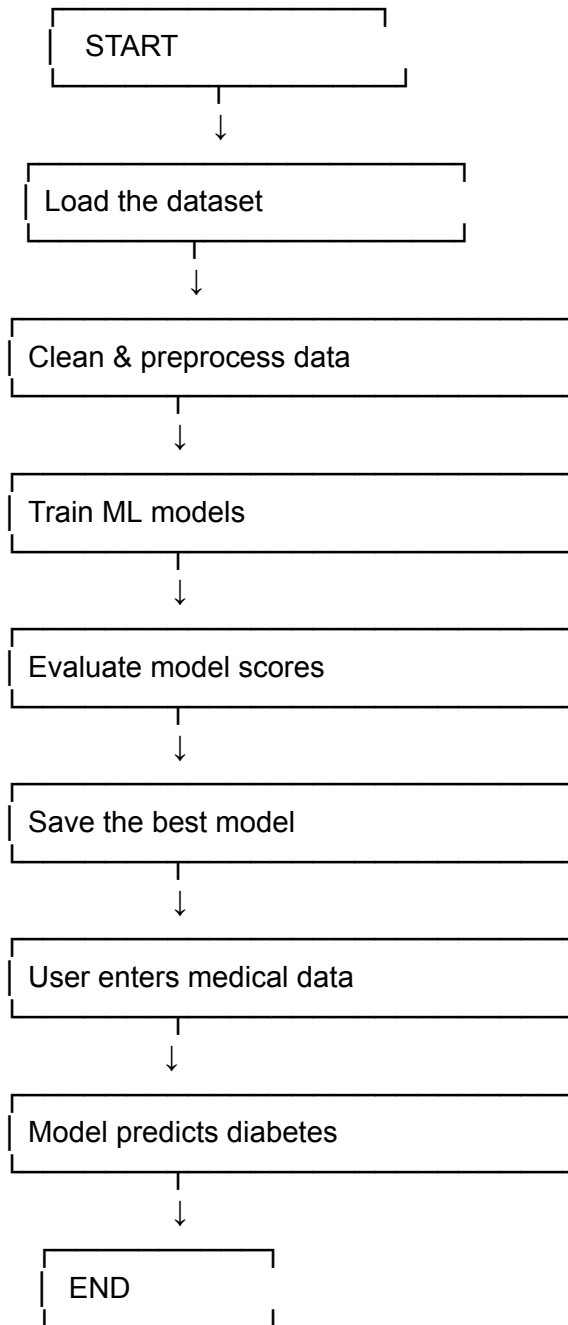
Step 9: Interactive Input

I added a small input-based system where a user can type medical values and get a diabetes prediction.

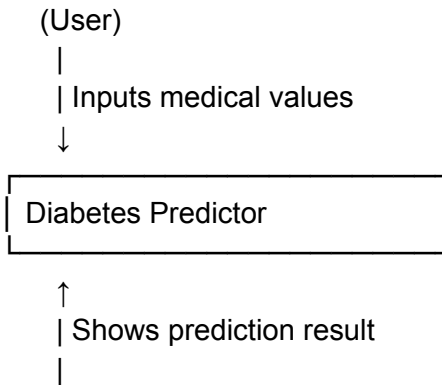
9. SYSTEM ARCHITECTURE DIAGRAM:



10. FLOWCHART:



11. USE-CASE DIAGRAM:



12. RESULTS:

Below are the evaluation results -

```
File Edit Selection View Go Run Terminal Help
C:\Users> samri > Downloads > diabetes detector.ipynb > Import as
Generate Code Markdown Run All Restart Clear All Outputs Jupyter Variables Outline
Python 3.12.10

Both models trained successfully.

# Quick helper function so I don't rewrite the same lines 20 times
def get_scores(model, X_test, y_test):
    preds = model.predict(X_test)
    probs = model.predict_proba(X_test)[:, 1] # probability of being diabetic

    scores = {
        "Accuracy": accuracy_score(y_test, preds),
        "Precision": precision_score(y_test, preds),
        "Recall": recall_score(y_test, preds),
        "F1 Score": f1_score(y_test, preds),
        "ROC AUC": roc_auc_score(y_test, probs)
    }

    return scores

log_results = get_scores(log_model, X_test_prepped, y_test)
rf_results = get_scores(rf_model, X_test_prepped, y_test)

print("Logistic Regression:", log_results)
print("Random Forest:", rf_results)

Logistic Regression: {'Accuracy': 0.7077922077922078, 'Precision': 0.6, 'Recall': 0.5, 'F1 Score': 0.5454545454545454, 'ROC AUC': 0.812962962962963}
Random Forest: {'Accuracy': 0.7402597402597403, 'Precision': 0.6666666666666666, 'Recall': 0.5185185185185185, 'F1 Score': 0.5833333333333334, 'ROC AUC': 0.8105555555555556}

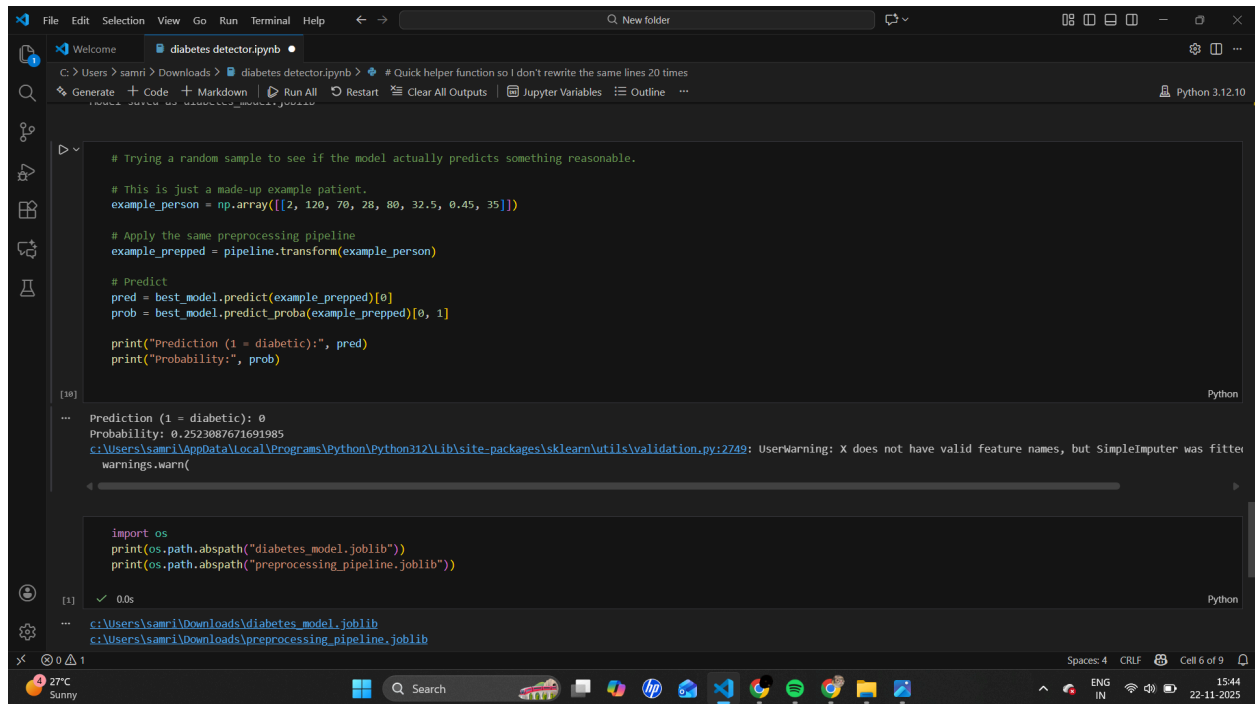
# I picked the model with the better ROC AUC score.
# In my case, RF usually wins, but I still checked.

best_model = rf_model if rf_results["ROC AUC"] > log_results["ROC AUC"] else log_model
```

Showing accuracy, precision, recall, F1 and ROC-AUC.

The Random Forest model slightly outperformed Logistic Regression due to its ability to handle non-linear relationships.

Next, the system's prediction output:



The screenshot shows a Jupyter Notebook titled "diabetes detector.ipynb" in a dark-themed environment. The notebook is open to a cell containing Python code for testing a model. The code includes comments and a sample patient data array. It uses a pre-trained pipeline to transform the data and then predicts the probability of a patient being diabetic. The output shows a prediction of 0 and a probability of approximately 0.25. A warning message from sklearn is also visible. Below this, another cell contains code to save the model and pipeline as joblib files. The output of this cell shows the file paths where the files were saved.

```
# Trying a random sample to see if the model actually predicts something reasonable.

# This is just a made-up example patient.
example_person = np.array([[2, 120, 70, 28, 80, 32.5, 0.45, 35]])

# Apply the same preprocessing pipeline
example_prepped = pipeline.transform(example_person)

# Predict
pred = best_model.predict(example_prepped)[0]
prob = best_model.predict_proba(example_prepped)[0, 1]

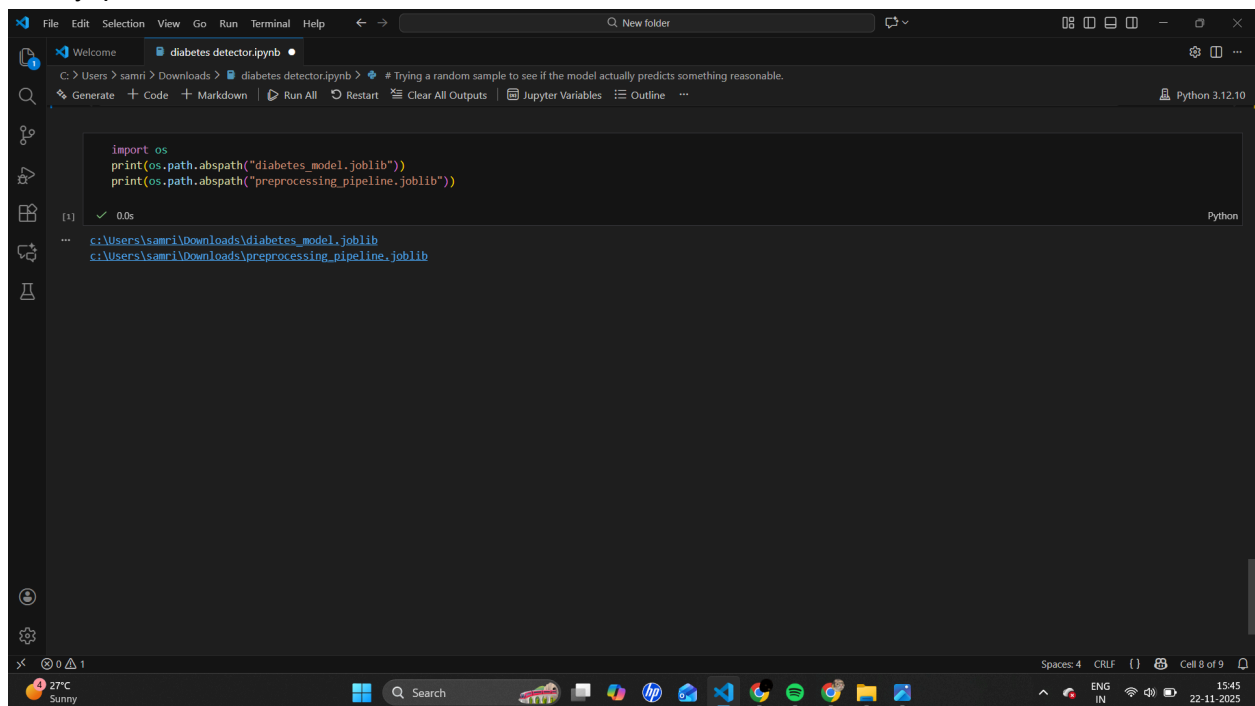
print("Prediction (1 = diabetic):", pred)
print("Probability:", prob)
```

```
[10] ... Prediction (1 = diabetic): 0
Probability: 0.2523087671691985
c:\Users\samri\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid feature names, but SimpleImputer was fitted with feature names
  warnings.warn(

import os
print(os.path.abspath("diabetes_model.joblib"))
print(os.path.abspath("preprocessing_pipeline.joblib"))
```

```
[1] ... c:\Users\samri\Downloads\diabetes_model.joblib
c:\Users\samri\Downloads\preprocessing_pipeline.joblib
```

Finally, proof that the model files were saved:



This screenshot shows the same Jupyter Notebook, but now the cell containing the file saving code is selected. The code is identical to the one in the previous screenshot. The output of this cell is displayed, showing the full file paths for "diabetes_model.joblib" and "preprocessing_pipeline.joblib" in the Downloads folder, confirming that the files were successfully saved.

```
import os
print(os.path.abspath("diabetes_model.joblib"))
print(os.path.abspath("preprocessing_pipeline.joblib"))
```

```
[1] ... c:\Users\samri\Downloads\diabetes_model.joblib
c:\Users\samri\Downloads\preprocessing_pipeline.joblib
```


13. CHALLENGES FACED:

- When to impute zeros in the dataset
- Pipelines and imputers
- Evaluation criteria
- Being unsure about Jupyter's directory
- Saving and loading ML models with joblib

14. LEARNINGS:

- Full ML project workflow: cleaning → preprocessing → training → evaluation
- Importance of data preprocessing
- How classification metrics work
- How to implement interactivity in notebooks
- How to compare ML models properly

15. FUTURE SCOPE:

- Deploy model as a web app using Streamlit
- Use advanced ML models like XGBoost
- Add more medical features for better accuracy
- Add charts and visualizations for data understanding
- Turn it into a mobile health screening tool

16. CONCLUSION:

This project successfully demonstrates how machine learning can assist with early diabetes prediction. By cleaning the dataset, preprocessing values, training ML models, and evaluating their accuracy, I built a complete and functional diabetes prediction system. The final interactive version allows users to enter medical measurements and instantly receive a prediction, showing how machine learning can help in real-world healthcare scenarios.